DHBW Stuttgart

Karsten Keßler

Datenbanksysteme

3. Entity-Relationship-Modell

Kurs Informatik TINF24ITA

Stand: 01.10.2026

DHBW Stuttgart

Übersicht der Gesamtvorlesung

Thema	Inhalte (Auszug)
1 Einführung	Einführung, Begriffe, Eigenschaften, Funktionen, Datenintegration
2 Datenbankmodelle u. –architekturen	ANSI/SPARC-Architektur, Verschiedene Datenbankmodelle, Entwicklung
3 ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
4 Relationales Modell	Relationales Datenbankmodell im Detail
5 Relationale Algebra	Operatoren, Anwendung, Nützlichkeit
6 Normalformen	Normalisierung von Datenbanken, funktionale Abhängigkeiten, Synthesealgorithmus
7 Transaktionen	ACID, Grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
8 SQL DML	Datenmanipulation mit SQL
9 SQL DDL	Datendefinition mit SQL
10 Integrität	Integrität im relationalen Modell wiederholt, Umsetzung (in SQL)
11 Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
12 Indices	Interne Strukturen, Nützlichkeit für Anfragen
13 Optimierung	Optimierung von Datenbanken als Thema für die Zusammenfassung der Vorlesung

DHBW Stuttgart

Inhalte der heutigen Vorlesung

- Übersicht
- Beziehungen
- Schlüssel
- Kardinalität
- Attribute
- Erweiterung
- Beispiel
- UML

Übersicht des Entwurfsprozesses

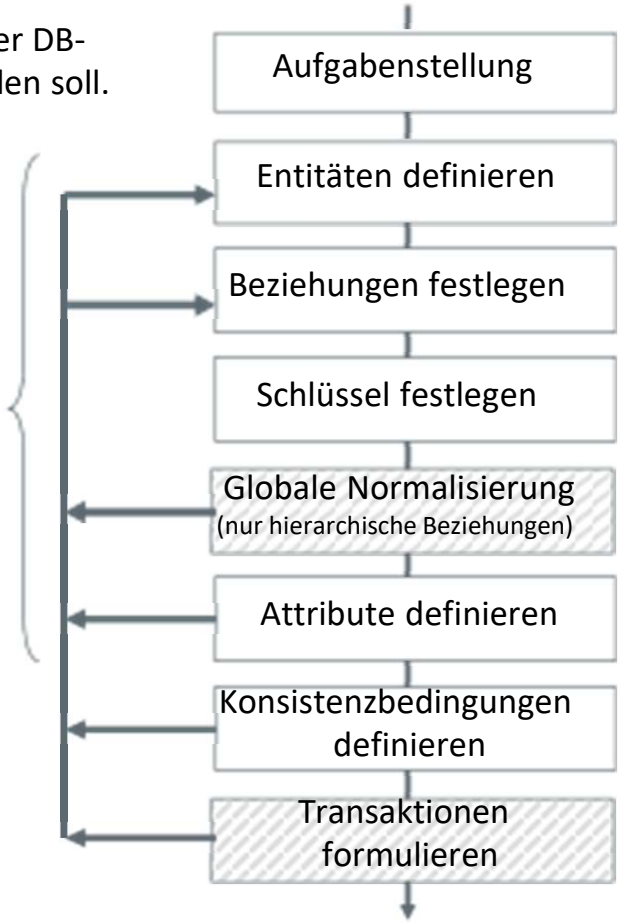
Anforderungsanalyse (Requirements)
Apps, Daten, Operationen

Konzeptionelle Ebene
ER-Modell (welche Daten gibt es? Welche Bedeutung haben sie?
Wie sehen die Geschäftsprozesse aus?

Implementations- bzw logische Ebene
Relationen, Tupel, Attribute: Wie werden die Daten strukturiert?
Welche Verknüpfungen gibt es zwischen den Daten?

Physische Ebene
Index, Performance, Hardware, Betriebssystem, welches DBMS?

Genaue Beschreibung was mit der DB-Anwendung geleistet werden soll.



Formulare/Abfragen

DHBW Stuttgart

Exkurs: Eigenschaften eines Modells

- Abbildungsmerkmal: bilden die Realität ab
- Pragmatisches Merkmal: haben einen Zweck
- Reduktionsmerkmal: lassen Teile der Realität weg
- Abundanzmerkmal: fügen etwas zur Realität zu



DHBW Stuttgart

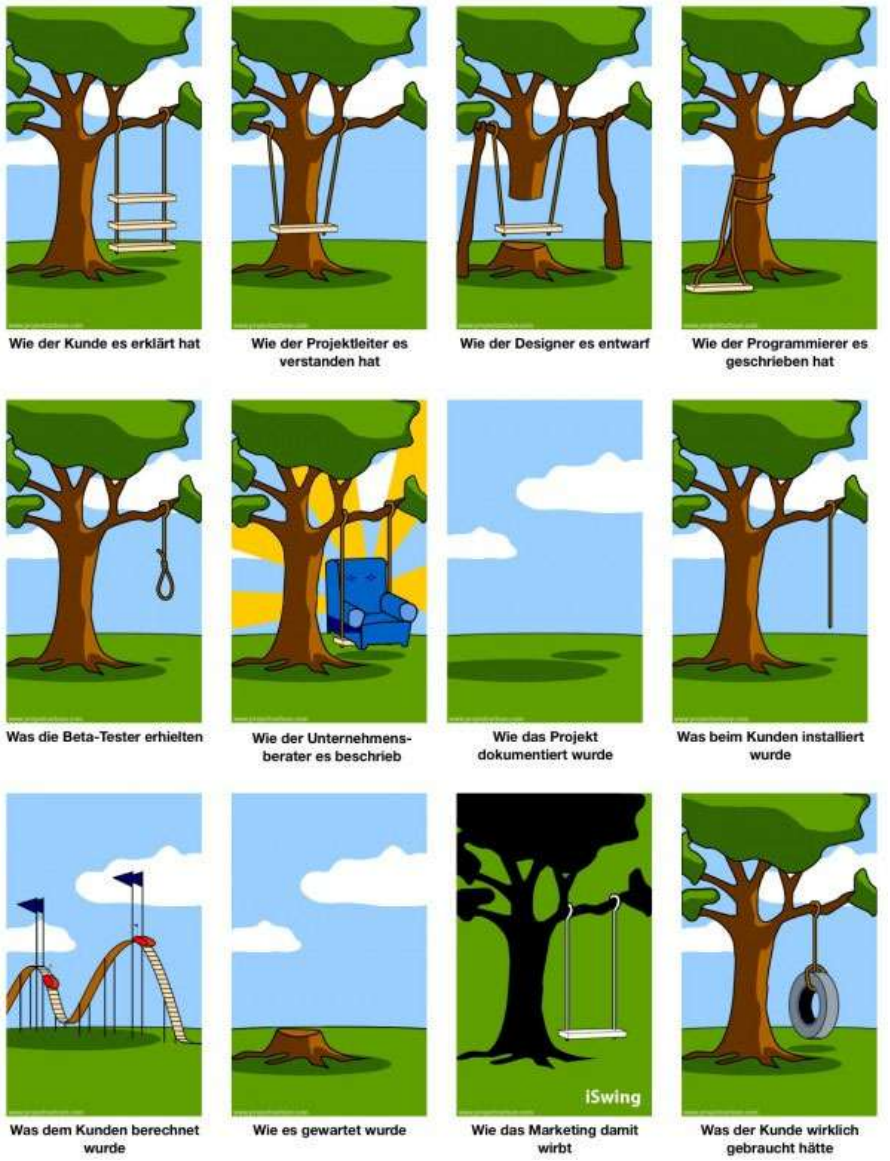
Konzeptueller Entwurf mit ERM

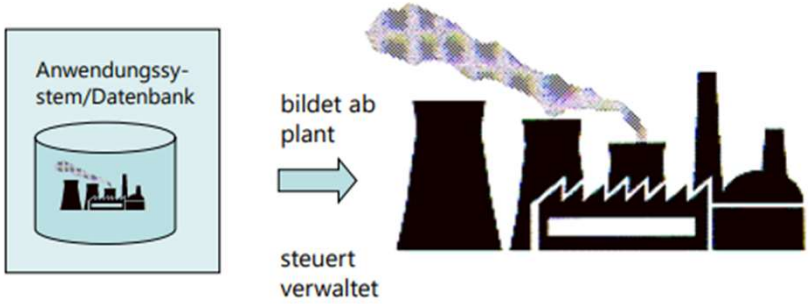
Die ideale Spezifikation ist vollständig, eindeutig, verständlich (für alle Leser), redundanzfrei.

Konzeptueller Entwurf = Modellierung der Daten → folgt der Anforderungsanalyse.

Ein **ER-Schema**

- Enthält alle Informationseinheiten, die im zu implementierenden DBS enthalten sein sollen (Entities, Relationships, Attribute).
- Dient auch der Kommunikation mit späteren Benutzern
- Kann (relativ) einfach (aufwandsarm) in ein logisches Schema transformiert werden
- Ist eine grafische Repräsentation des konzeptuellen Modellierung der Daten
- Ist ein individuelles Ergebnis





Das Entity-Relationship-Modell

„Gegenstands-Beziehungs-Modell“

Entity
Entität, Gegenstand der realen Welt. Von einer anderen Entität unterscheidbar.
Konkretes oder abstraktes Objekt, über welches Informationen gespeichert werden.
Z. B.: Buch, Vorlesung, Prüfung
$E_1, E_2, \ldots, E$n

Buch

Beziehung
Relationship / Beziehung

Beziehung zwischen zwei oder mehreren Entitäten

Z. B.: Person (Entität) leiht aus (Relationship) ein Buch (Entität)

$R \subseteq E_1 \times E_2 \times \ldots E_n$

Normalerweise n=2 oder 3.

ausleihen

Attribut
Attribut, Eigenschaft

Eigenschaften von einer Beziehungen oder Entitäten

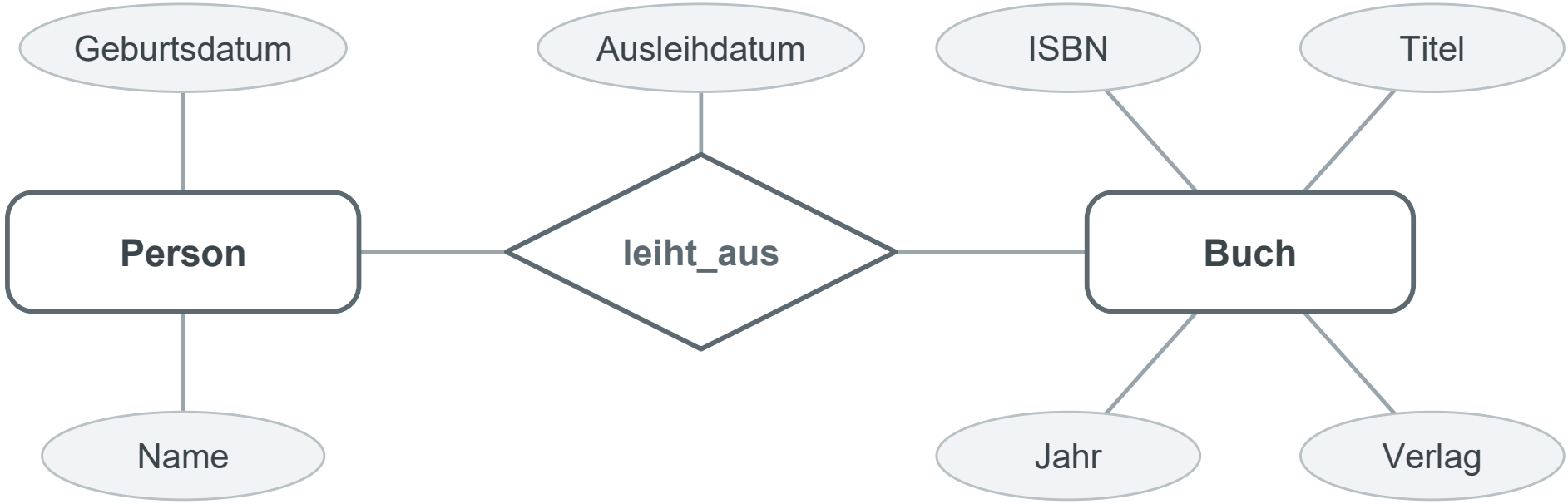
Wertebereich bestimmt Zulässigkeit bestimmter Werte (Zahl, Zeichen, usw)

z. B. Buchtitel, Name, ..

Buchtitel

„klassisches ER Modell“ nach P. P. Chen 1976, heute Standardmodell für (frühe) Entwurfsphasen der Datenbankentwicklung, konzeptueller Entwurf

Bibliotheksbeispiel

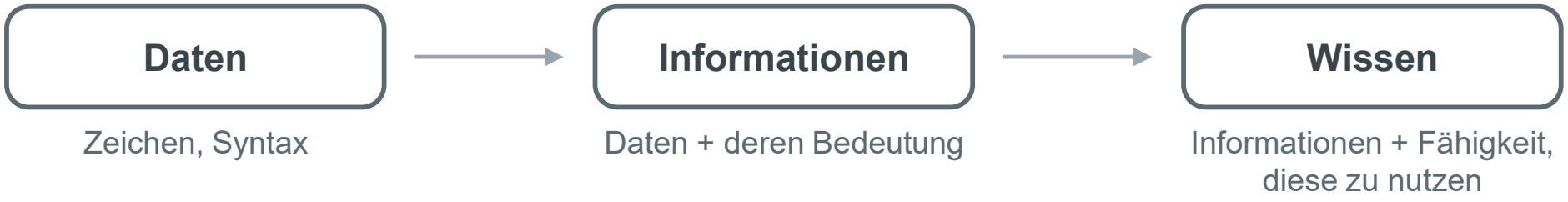


DHBW Stuttgart

Warum der Aufwand dieser Datenmodellierung?

- erhöht die Transparenz des Programmcodes
- reduzieren den Aufwand für Wartung und Weiterentwicklung der Applikationen
- reduziert den Integrationsaufwand bei Zusammenarbeit mit externen Softwarekomponenten
- bildet Grundlage bei Diskussionen zwischen Fach- und Entwicklungsabteilungen
- dient zur Abstraktion betrieblicher Prozesse und deren Optimierung

VHIT-Methode:
Vom Hirn ins Terminal



DHBW Stuttgart

Mögliche Vorgehensweise bei der Datenmodellierung

1. Identifizieren Sie die Entitätstypen:
   Analysieren Sie den Anwendungsfall nach entsprechenden Typen, die auftauchen. Z.B. „Fahrzeuge“, „Verkäufer“, „Filialen“, „Kunden“

2. Identifizieren Sie die Beziehungstypen:
   Analysieren Sie den Anwendungsfall nach Beziehungen zwischen den Entitäten. Z.B. „arbeitet in“ oder „fordert an“.
   Erstellen Sie ein Rohentwurf des Datenmodells.

3. Ergänzen Sie Attribute und entsprechende (unterstrichene) Schlüsselattribute:
   Analysieren Sie die Entitäten nach ihren Attributen bzw. Schlüsselattributen.

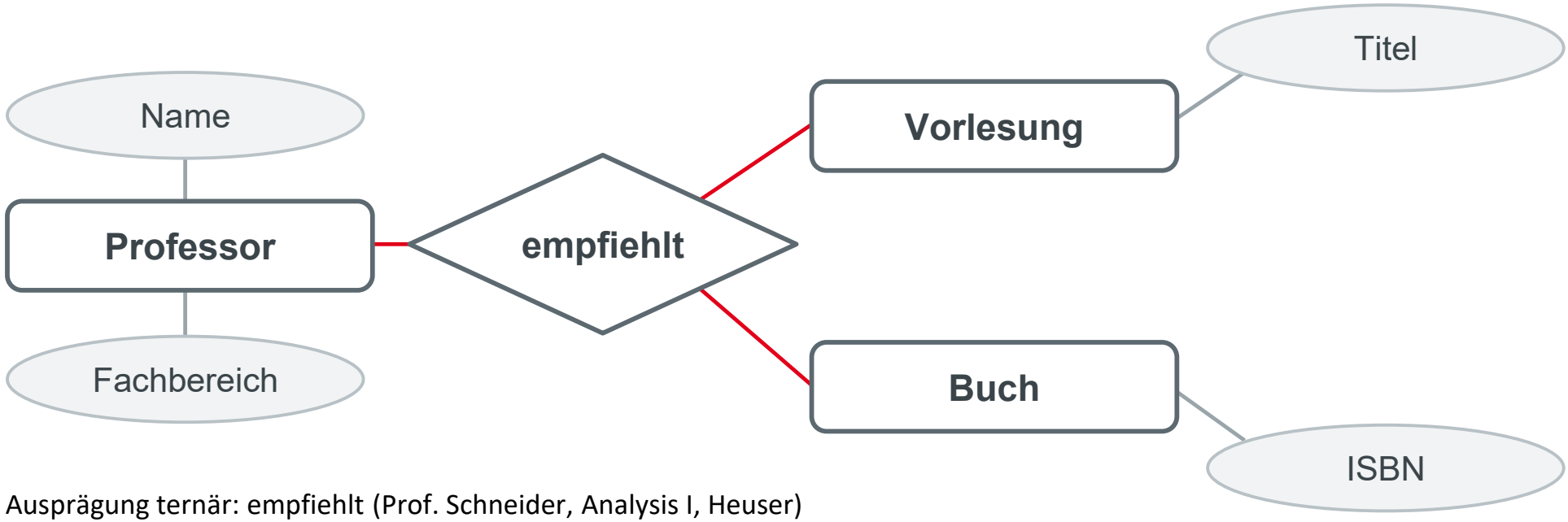
- Entitäten werden aus den Geschäftsprozessen und Prozessen abgeleitet
- Beziehungen brauchen Kardinalitäten, erst dann ist das Diagramm vollständig
- Modell sollte von Fachexperten validiert werden

→ Umwandlung in ein physisches Modell

DHBW Stuttgart

Beziehungen im Entity-Relationship-Modell

Vergleich: 3-stellige und 2-stellige Beziehungen (3 Entitätstypen)

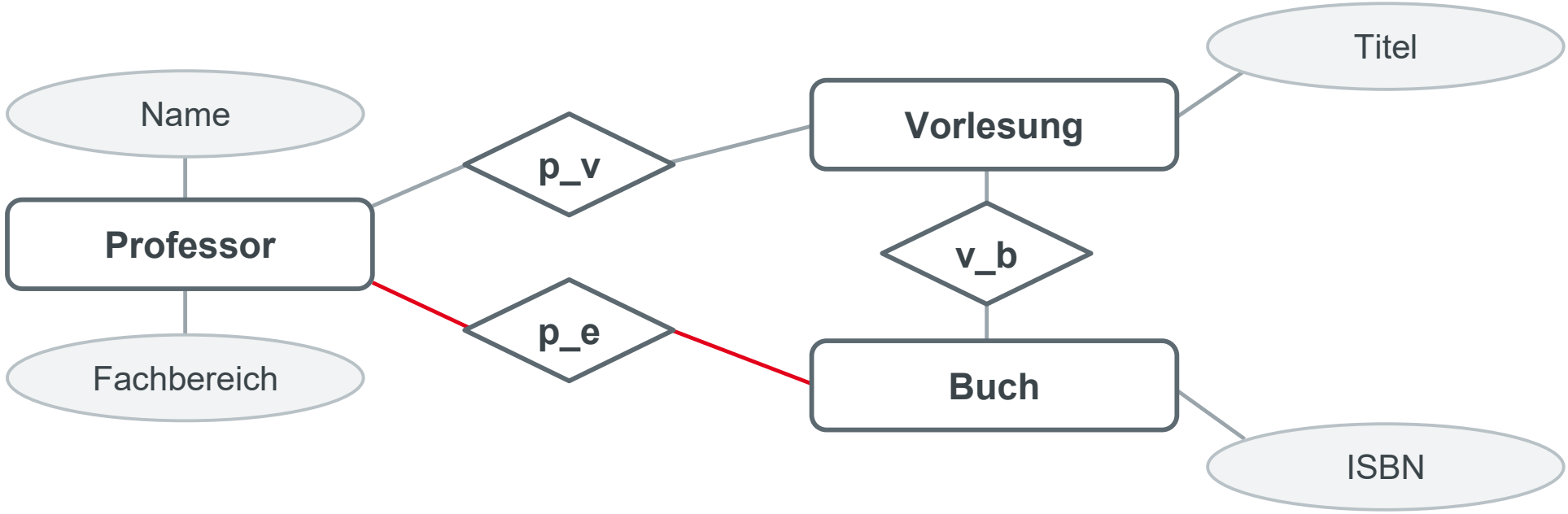


Ausprägung ternär: empfiehlt (Prof. Schneider, Analysis I, Heuser)
empfiehlt (Prof. Kurz, Analysis II, Heuser)
empfiehlt (Prof. Schmitt, Analysis II, Wenisch)

DHBW Stuttgart

Beziehungen im Entity-Relationship-Modell

Vergleich: 3-stellige und 2-stellige Beziehungen (2 Entitätstypen)



Ausprägung binär: prof_empfiehlt (Prof. Paul, Lineare Algebra II)

Eine ternäre Beziehung würde mindestens drei Entitätstypen auf einmal verbinden. Hier gibt es jedoch keine Verbindung, die die drei Entitätstypen gleichzeitig miteinander in einer einzigen Beziehung darstellt. Stattdessen bestehen hier mehrere getrennte binäre Beziehungen zwischen den Entitäten. Dabei geht allerdings verloren, dass alle drei Beziehungen zusammengehören.

Beziehungen im Entity-Relationship-Modell

Unterschiedliche Rollen/Rekursive Beziehungen zwischen gleichartigen Entitätstypen

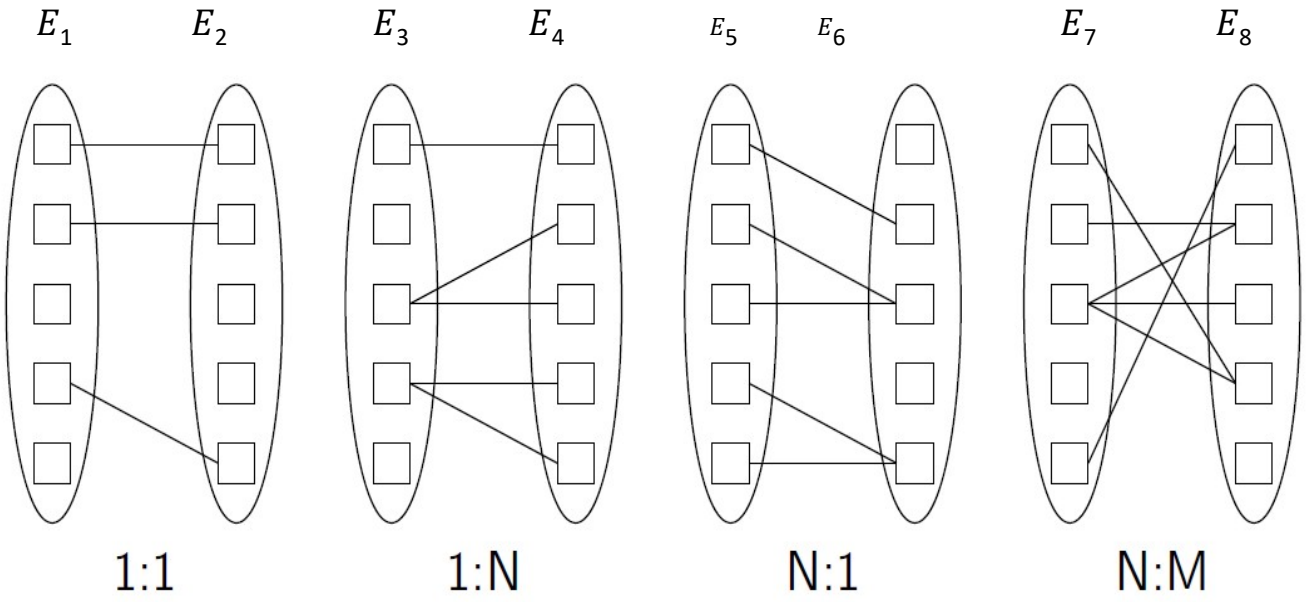


Mitarbeiter_ID	Name	Position	Chef_ID
1	Lisa Müller	Abteilungsleiter	NULL
2	Peter Schmidt	Teamleiter	1
3	Anne Maier	Mitarbeiter	2
4	Max Hose	Mitarbeiter	2

Hierarchie: ein Mitarbeiter (z.B. Teamleiter) kann n Mitarbeiter führen und gleichzeitig Untergebener sein (z.B. von einem Abteilungsleiter)

Kardinalitäten

- Beziehungen zwischen Entitätstypen können einfacher oder komplexer Natur sein.
  Kardinalitäten drücken aus, mit wieviel anderen Entitäten eine Entität eine Beziehung hat.



Kardinalitäten/Minimal-Kardinalitäten

Kardinalitäten hängen direkt
vom Datenmodell des Benutzers ab!

Standard ER-Modell



1 : 1



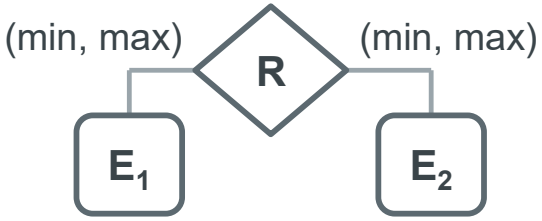
Standard ER-Modell
mit Kardinalitäten



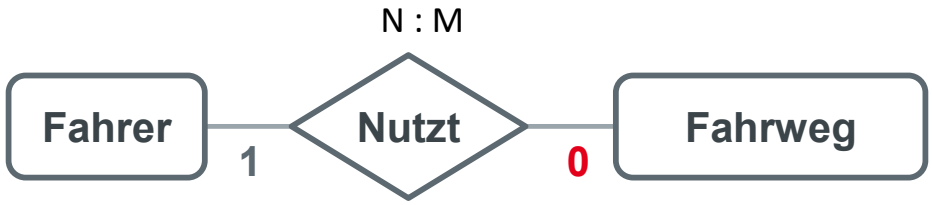
1 : N



(min, max) Notation



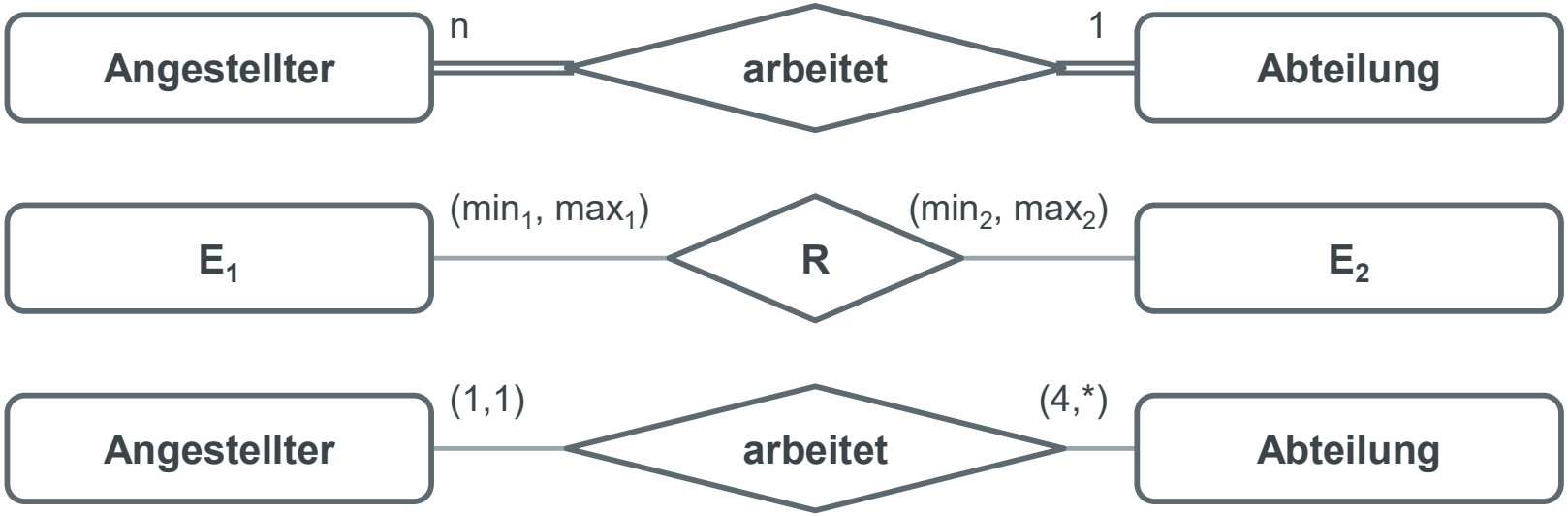
N : M



Minimalkardinalitäten

Alternative Diagrammnotation: (min, max)

Totale Partizipation: jedes Element muss in Beziehung stehen („Doppelstrich“):
jeder Angestellte muss einer Abteilung zugeordnet sein und jede Abteilung muss mindestens einen Angestellten haben)



(min, max)-Notation nicht intuitiv, Leserichtung umgekehrt zu Chen: z.B: „Abteilung steht in Beziehung mit mindestens 4 Angestellten“

Diese alternative Notation wird oft verwendet, um klarere Spezifikationen für minimale Anforderungen in Beziehungen darzustellen und bietet präzisere Informationen über die Struktur der Daten.

DHBW Stuttgart

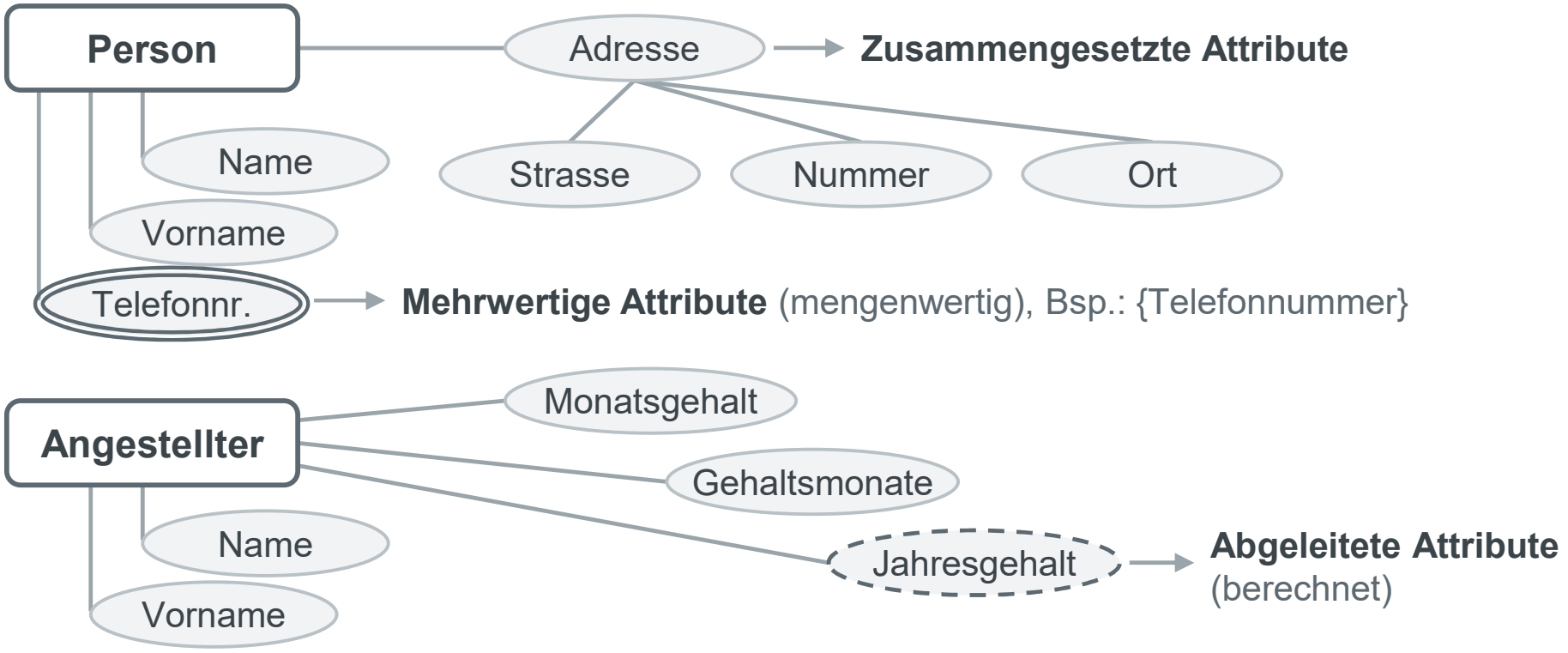
Attribute

- Zusammengesetzte Attribute: setzen sich aus mehreren Einzelattributen zusammen
  Beispiel: Adresse setzt sich aus Land, PLZ, Ort, Straße und Hausnummer zusammen

- Abgeleitete Attribute: lassen sich direkt aus anderen Werten ermitteln. Müssen also nicht zwingend in der Datenbank abgespeichert sein.
  Beispiel: das Attribut Alter eines Mitarbeiters lässt sich aus dem Attribut Geburtsdatum und Tagesdatum ermitteln

- Mehrwertige Attribute: normalerweise bestehen Attribute nur aus einem Wert (einwertige Attribute). Manche Attribute können jedoch mehrere Werte zum gleichen Zeitpunkt annehmen.
  Beispiel: das Attribut Hochschulabschluss kann die Werte Bachelor, Master und Doktor annehmen oder auch keinen Wert.

Attribute


Person
Adresse
Zusammengesetzte Attribute
Name
Strasse
Nummer
Ort
Vorname
Telefonnr.
Mehrwertige Attribute (mengenwertig), Bsp.: {Telefonnummer}
Angestellter
Monatsgehalt
Gehaltsmonate
Name
Jahresgehalt
Abgeleitete Attribute
(berechnet)
Vorname

Schlüssel im ERM

Einige Attribute bilden mit ihren Werten bereits **eine eindeutige Identifizierung** für Entitäten eines Entitätstyps.

Beispiele: Student **Matr.Nr.**

Buch **ISBN**

Angestellter p**ersonal.Nr.**

Schlüssel	Erklärung	Beispiel
Superschlüssel	Eine Attributmenge heißt Superschlüssel von E, wenn sie jede Entität eindeutig identifiziert	{MatrNr} , {Email}, {MatrNr, Name} {Email, Studiengang, GebDatum}
Kandidatenschlüssel	Ein **minimaler** Superschlüssel	{MatrNr} {Email}
Primärschlüssel	Ein **ausgewählter** Kandidatenschlüssel, der als der Schlüssel des Entitätstyps dient	{MatrNr} {Email} ist Alternativschlüssel

Eine solche minimale Attributmenge heißt Schlüssel (ohne Minimalität: Superschlüssel).

Verschiedene mögliche Schlüssel: **Schlüsselkandidaten** -> Auswahl eines **Primärschlüssels**

Dieser Primärschlüssel wird unterstrichen E(..., $\underline{S_1}$, ..., ...)

Formal: Entitätstyp E(A_1,...,A_n) mit Schlüsselattributen { S_1, ..., S_m} $\subseteq$ {A_1, ..., A_n}

→ Der Entitätstyp E mit den Attributen A_1,..., A_n enthält eine Schlüsselattributmenge {S1, ..., Sm}, die Teilmenge aller Attribute ist. Jede minimale solche Menge ist ein Schlüsselkandidat, einer wird Primärschlüssel.

Existenzabhängige Entitätstypen im ERM

Schwache Entitäten können nicht autonom existieren und sind von einer übergeordneten Entität abhängig. Jede schwache Entität muss daher einer starken Entität zugeordnet sein, da sie ohne diese nicht existieren kann.

Identifizierende Beziehung als Schlüssel für abhängige Entitätstypen

Starke Entitätstypen (mit eigenem Schlüssel) vs. **Schwache** Entitätstypen (kein eigener Schlüssel, nur partieller)

Existenzabhängigkeit ⇒ totale Teilnahme: jedes Buchexemplar muss zu einem Buch gehören (Doppelstrich)

Meist 1:N/N:1, seltener 1:1-Beziehung

Nummer: laufende Exemplarnummer, nur je Buch eindeutig (partieller Schlüssel, gestrichelt); eindeutig erst ISBN + Numr

ISBN identifiziert das Buch eindeutig (starke Entität)



Bsp. Bibliothekssystem: Jedes Buch hat mehrere Exemplare; ohne Buch gibt es auch keine Exemplare mehr.

DHBW Stuttgart

Existenzabhängige Entitätstypen im ERM: Pfeilnotation

Alternative Darstellung nach Saake/Sattler/Heuer 2018

„Funktionale Beziehung“

Identifizierende Beziehung
(schwache Entität von der starken abhängig)

Partieller Schlüssel
(Notation wie S. 20)

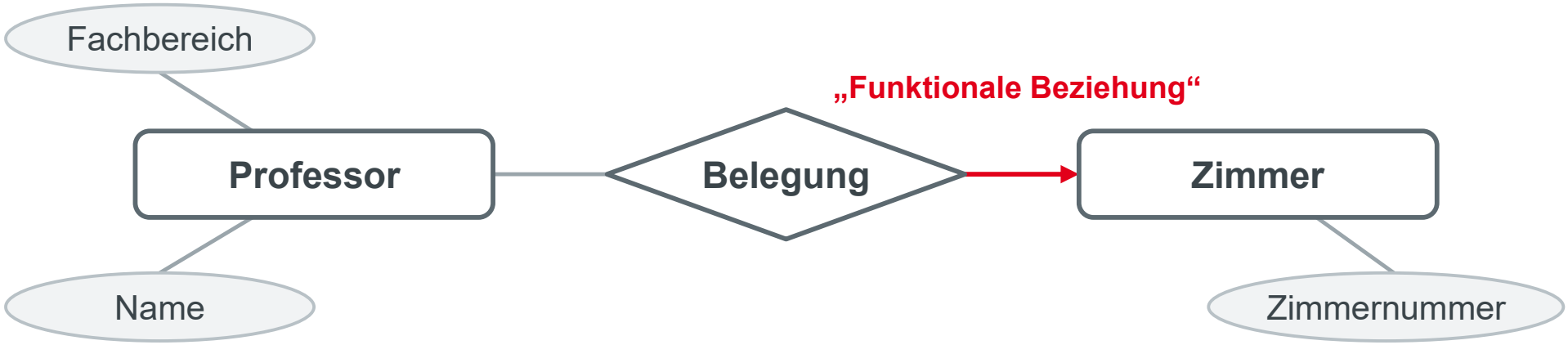


„gehört_zu“ ist eine funktionale Beziehung: Der Pfeil zeigt auf Buch, weil jedes Buchexemplar genau einem Buch zugeordnet ist. Nur über diese Beziehung lässt sich die schwache Entität identifizieren.

DHBW Stuttgart

Allgemein nach Saake/Sattler/Heuer: Funktionale Beziehungen

„Jeder Entität eines Entitätstyps E1 wird **maximal eine** Entität eines Entitätstyps E2 zugeordnet.“



Richtung der **funktionalen Abhängigkeit** explizit durch einen Pfeil: jedem Professor ist höchstens ein Zimmer zugeordnet, d. h. Zimmer ist funktional abhängig von Professor (Professor → Zimmer).

Ausblick: Implementierung schwacher Entitäten

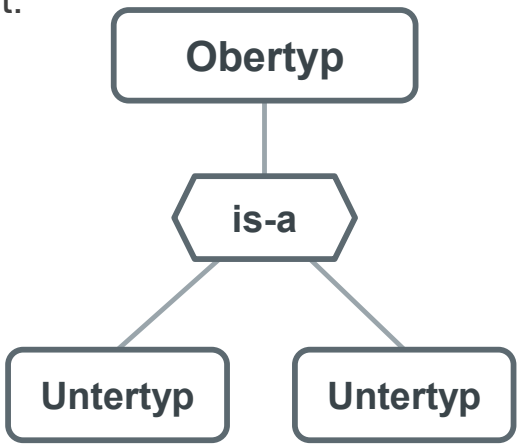
```
CREATE TABLE dbo.Buch (
  ISBN  char(13)    NOT NULL,
  Titel nvarchar(200) NOT NULL,
  CONSTRAINT PK_Buch PRIMARY KEY (ISBN)
);

CREATE TABLE dbo.Buchexemplar (
  ISBN    char(13)     NOT NULL,  -- FK-Teil und PK-Teil
  Nummer  int          NOT NULL,  -- partieller Schlüssel (nur je Buch eindeutig)
  Standort nvarchar(50) NULL,
  Zustand  nvarchar(20) NULL,
  CONSTRAINT PK_Buchexemplar PRIMARY KEY (ISBN, Nummer),
  CONSTRAINT FK_Buchexemplar_Buch
    FOREIGN KEY (ISBN) REFERENCES dbo.Buch(ISBN)
    ON DELETE CASCADE      -- Existenzabhängigkeit
    ON UPDATE CASCADE      -- optional, falls ISBN änderbar wäre
);
```

Spezialisierung/Generalisierung (is-a) und Aggregation (is-part-of)

- **Spezialisierung (top-down):** Aus einem Obertyp werden Untertypen gebildet. Bsp.: Sekretärin, Ingenieur und Chef sind Spezialisierungen von Mitarbeiter.
- **Generalisierung (bottom-up):** Verschiedene Typen mit gemeinsamen Eigenschaften werden zu einem Obertyp zusammengefasst. Bsp.: Fahrrad und Auto werden zu Fahrzeug generalisiert.

Der Unterschied liegt nur in der Entwurfsrichtung. In beiden Fällen können Untertypen gegenüber dem Obertyp zusätzliche Attribute oder Beziehungen haben.



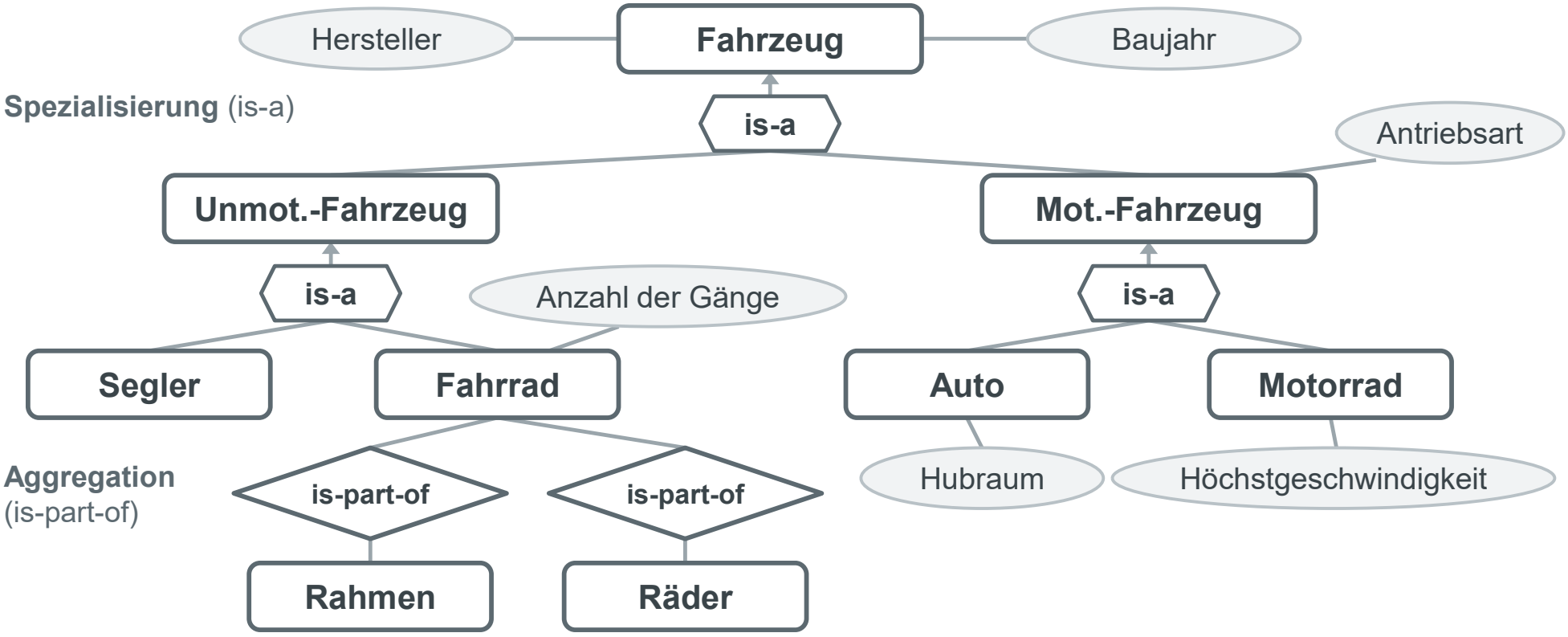
Wichtige Sonderformen (weitere Fälle werden nicht betrachtet):

- **Überlappend:** Eine Entität darf mehreren Untertypen angehören (ein Fahrzeug kann Elektrofahrzeug und SUV zugleich sein).
- **Disjunkt** ($A \cap B = \emptyset$)**:** Eine Entität gehört höchstens einem Untertyp an (die Untertypen schließen sich aus).
- **Total (davon unabhängige 2. Dimension):** Jede Entität gehört mindestens einem Untertyp an (jeder Mitarbeiter ist Sekretärin, Ingenieur oder Chef). Total heißt „mindestens einer“, nicht zwingend alle.

DHBW Stuttgart

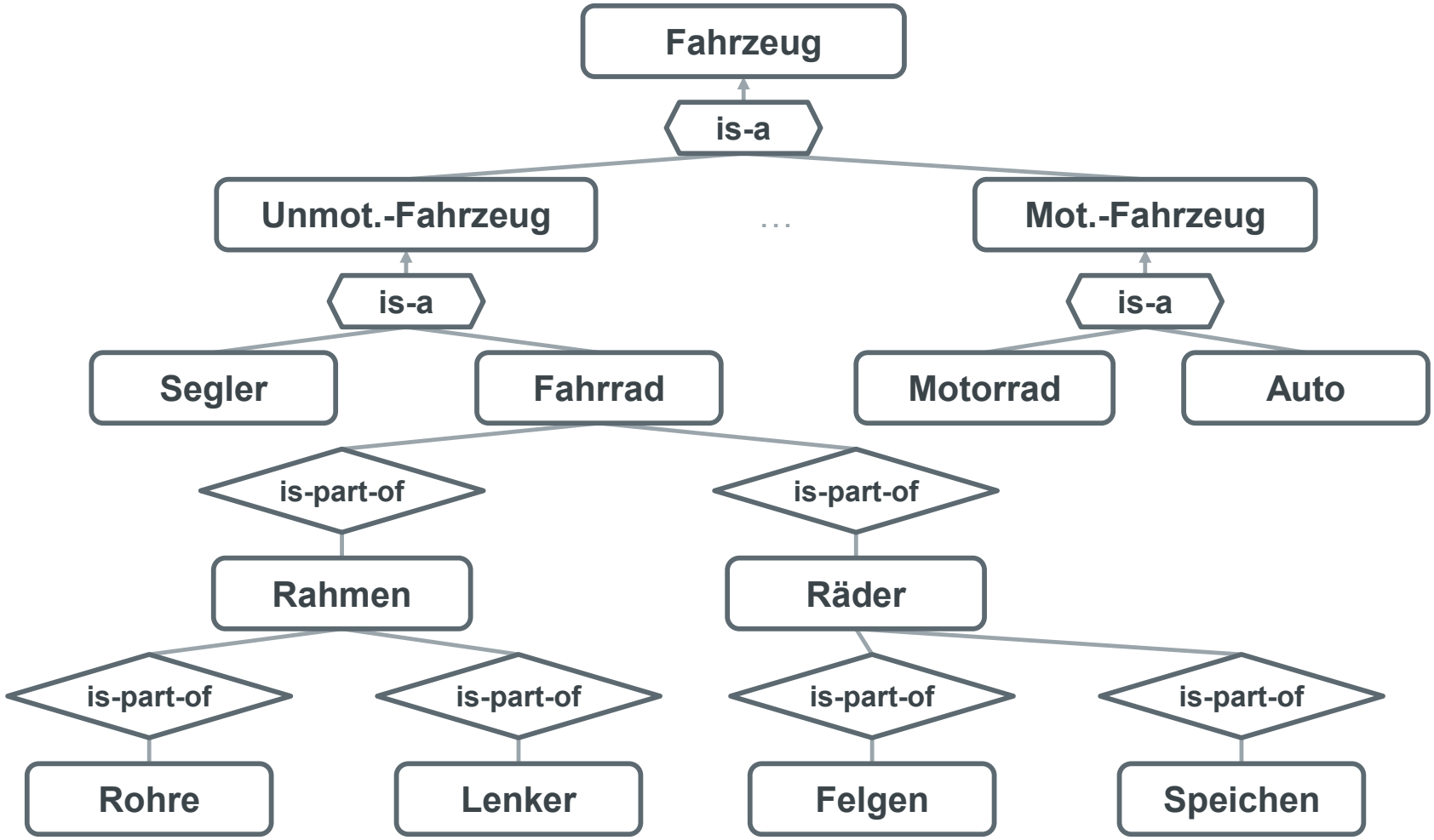
Beispiel: Spezialisierung und Aggregation

- **Aggregation (is-part-of):** Unterschiedliche Entitätstypen, die zusammen einen strukturierten Objektobertyp ergeben, werden einander zugeordnet.



Komposition und Aggregation (is-part-of)

- **Komposition:** Rahmen und Räder existieren nicht ohne das Fahrrad (Existenzabhängigkeit).
- **Aggregation:** Die Teile existieren auch ohne das Ganze (keine Existenzabhängigkeit).



is-a und is-part-of im Vergleich

	is-a	is-part-of
Leitfrage	Welcher Art ist es?	Woraus besteht es?
Test-Satz	„Jedes X ist ein Y.“	„X besteht aus Y.“
Objekte	ein Objekt, mehrere Typen	mehrere verschiedene Objekte
Vererbung	ja, Subtyp erbt Attribute	nein
Kardinalität	keine	ja (z. B. Fahrrad 1 : 2 Räder)
Beispiel	Fahrrad ist ein Fahrzeug Student ist eine Person	Rad ist Teil eines Fahrrads

Merksatz

Passt „ist ein“, ist es **is-a**. Passt „ist Teil von“ oder „besteht aus“, ist es **is-part-of**.
Gegenprobe 1: „Ein Rad ist ein Fahrrad“ ist falsch, also ist Rad zu Fahrrad kein is-a.
Gegenprobe 2: „Ein Auto besteht aus Fahrzeugen“ ist falsch, also Auto zu Fahrzeug kein is-part-of.

Implementierung von is-a und is-part-of in SQL

is-a (Generalisierung/Vererbung)

```sql
CREATE TABLE dbo.Fahrzeug (
  FahrzeugID  int IDENTITY PRIMARY KEY,
  Typ         varchar(20) NOT NULL CHECK
    (Typ IN ('Fahrrad','Motorrad','Automobil')),
  Bezeichnung nvarchar(100) NOT NULL
);

-- Ein Fahrrad ist ein Fahrzeug: PK = FK
CREATE TABLE dbo.Fahrrad (
  FahrzeugID int PRIMARY KEY
    REFERENCES dbo.Fahrzeug(FahrzeugID)
    ON DELETE CASCADE,
  AnzahlGaenge int NOT NULL
    CHECK (AnzahlGaenge > 0)
);

CREATE TABLE dbo.Motorrad (
  FahrzeugID int PRIMARY KEY
    REFERENCES dbo.Fahrzeug(FahrzeugID)
    ON DELETE CASCADE,
  Hubraum_cc int NOT NULL CHECK (Hubraum_cc > 0)
);
```

is-part-of (Komposition) und schwache Entität

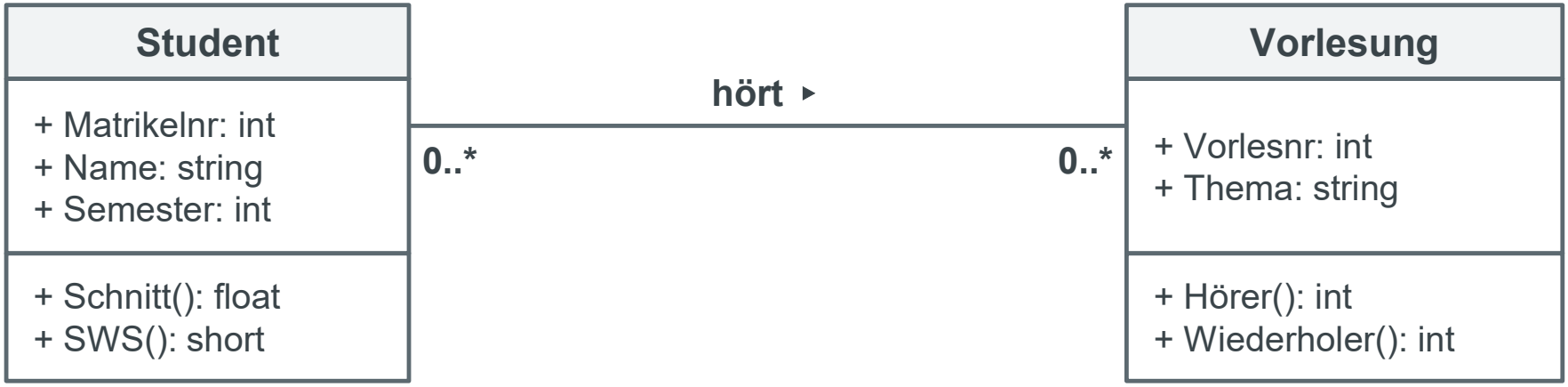
```sql
CREATE TABLE Bestellung (
  BestellNr int PRIMARY KEY,
  Kunde     nvarchar(100) NOT NULL,
  Datum     date NOT NULL
);

-- Eine Position ist Teil einer Bestellung:
-- ohne Bestellung keine Position
CREATE TABLE Position (
  BestellNr int NOT NULL,
  PosNr     int NOT NULL,
  ArtikelID int NOT NULL,
  Menge     int NOT NULL CHECK (Menge > 0),
  CONSTRAINT PK_Position
    PRIMARY KEY (BestellNr, PosNr),
  CONSTRAINT FK_Position_Bestellung
    FOREIGN KEY (BestellNr)
    REFERENCES Bestellung(BestellNr)
    ON DELETE CASCADE
);
```

is-a: Der Untertyp erbt den Schlüssel des Obertyps → PK = FK.
is-part-of: Das Teil hängt vom Ganzen ab → FK (NOT NULL) und zusammengesetzter PK.

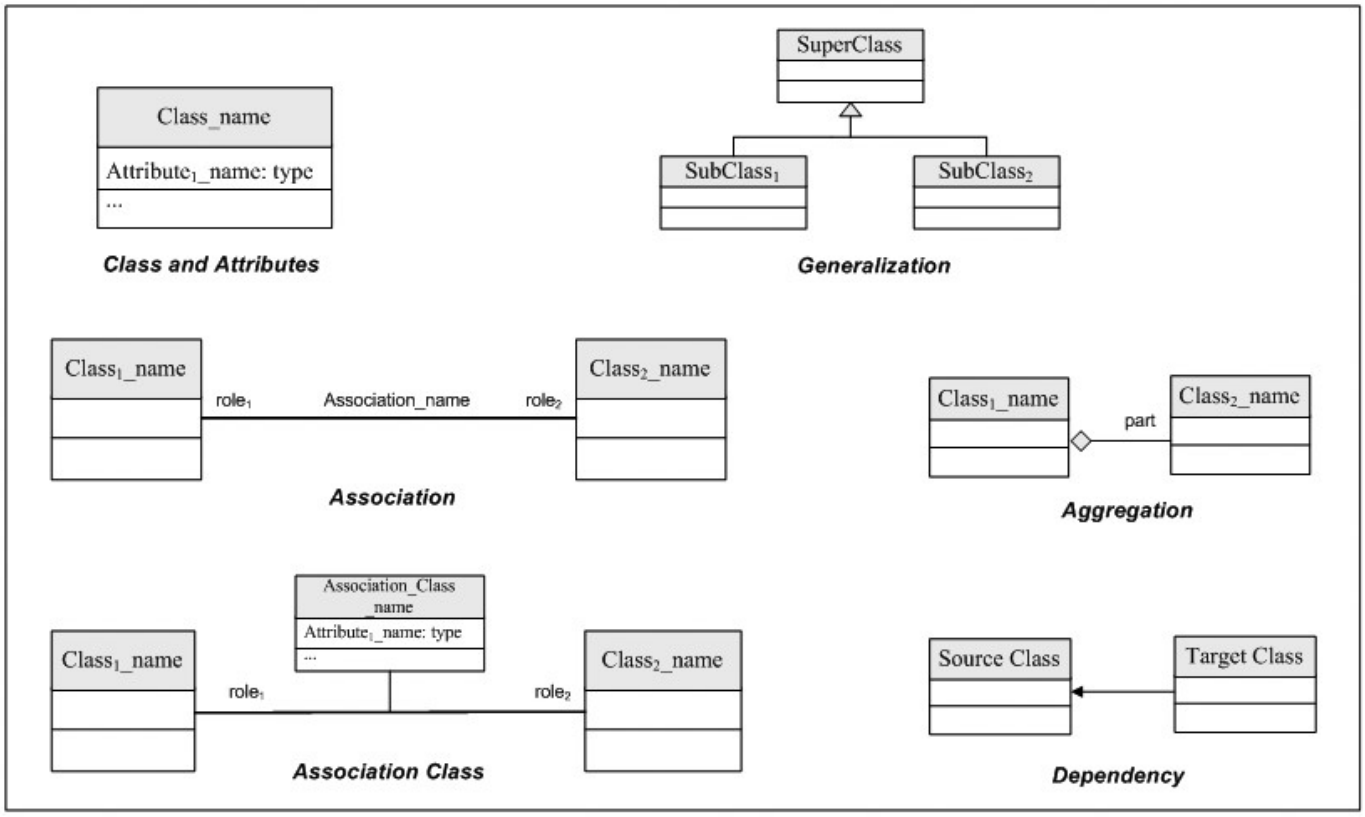
UML-Notation für das ERM

- **Attribute:** stehen im Kasten der Klasse.
- **Beziehungen:** Linie ohne Symbol; leere Raute = Aggregation, gefüllte Raute = Komposition.
- **Vererbung (is-a):** Linie mit leerem Dreieck, das auf die Oberklasse zeigt.
- **Methoden:** werden angegeben, für die DB-Modellierung aber nicht nötig.
- **Achtung:** Die Multiplizität gilt für das gegenüberliegende Ende (0..* bei Student: eine Vorlesung hat 0..* Hörer).



DHBW Stuttgart

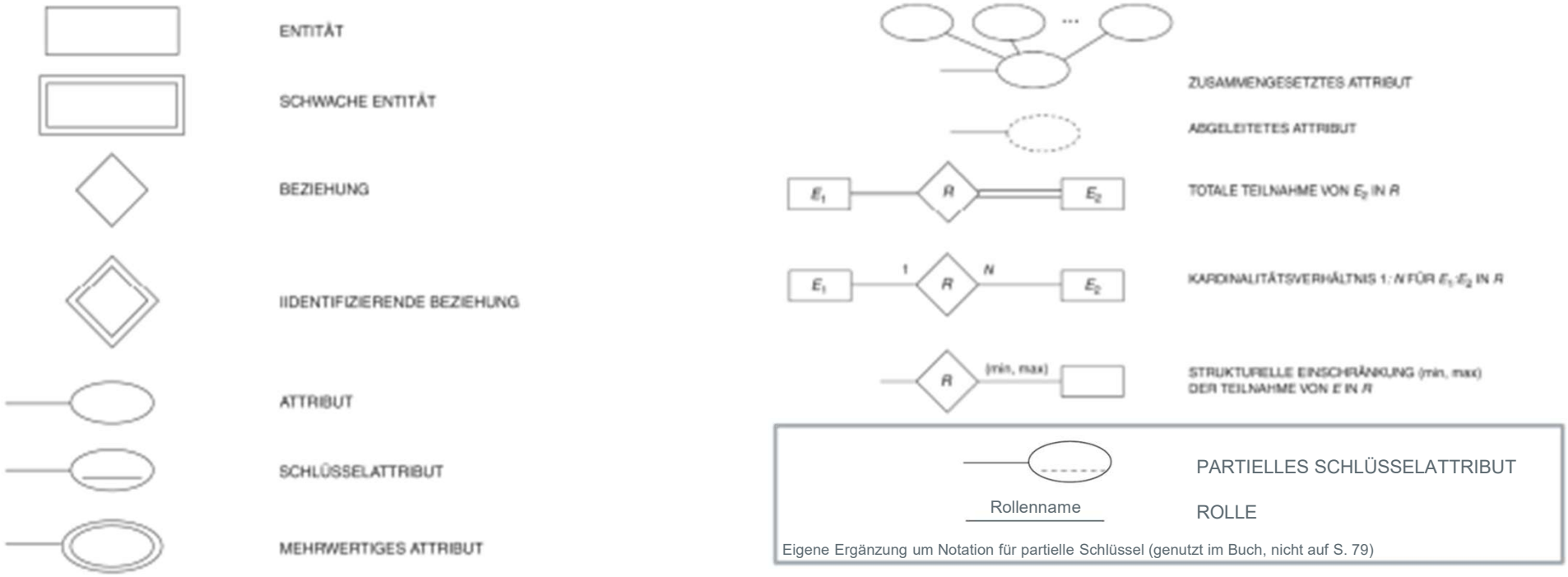
UML Review



*Bildquelle: Tong, Qiang; Zhang, Fu; Chen, Jingwei (2014): Construction of RDF(S) from UML Class Diagrams. In: *CIT* 22 (4), S. 237. DOI: 10.2498/cit.1002459.

DHBW Stuttgart

Anhang: ER-Diagrammnotation nach Elmasri/Navathe



Eigene Ergänzung um Notation für partielle Schlüssel (genutzt im Buch, nicht auf S. 79)

Bildquelle: Elmasri, Ramez; Navathe, Sham (2009): Grundlagen von Datenbanksystemen. Bachelorausgabe. 3. aktualisierte Auflage. München: Pearson Deutschland; Pearson Studium (Pearson Studium - IT)., S. 79,

Der Unterschied zwischen UML (Unified Modeling Language) und ER-Modellierung (Entity-Relationship-Modellierung) liegt hauptsächlich in ihrem Anwendungsbereich und ihrer Funktion:

Anwendungsbereich:

1. **ER-Modellierung**: Wird hauptsächlich im Bereich der Datenbanksysteme verwendet. Sie dient dazu, Datenstrukturen zu beschreiben, wie sie später in den Tabellen einer relationalen Datenbank abgebildet werden sollen.
2. **UML**: Wird im Software Engineering verwendet und beschreibt nicht nur Datenstrukturen, sondern auch Prozesse, Systemteilnehmer und deren Interaktionen. UML ist vielseitiger und kann zur Modellierung von Klassen, Abläufen, Zuständen und mehr verwendet werden.

Funktion:

1. **ER-Modellierung**: Konzentriert sich auf die Darstellung von Entitäten (Objekten) und deren Beziehungen. Es ist ein konzeptionelles Modell, das die logische Struktur einer Datenbank beschreibt.
2. **UML**: Bietet eine Vielzahl von Diagrammtypen, darunter Klassendiagramme, Anwendungsfalldiagramme, Sequenzdiagramme und Zustandsdiagramme. Diese Diagramme helfen bei der Visualisierung und Dokumentation verschiedener Aspekte eines Softwaresystems.

→ ER-Modellierung ist spezifisch für die Datenbankmodellierung, während UML ein umfassenderes Werkzeug für die Modellierung verschiedener Aspekte von Softwaresystemen ist.

Übungsaufgabe 1: ERM für ein Flugbuchungssystem

Erstellen Sie ein ERM für ein Buchungssystem einer Fluggesellschaft. Folgende Annahmen sind zu beachten:

- Ein Flug hat eine eindeutige Flugnummer und eine Abflugzeit.
- Ein Flugsteig hat eine eindeutige Nummer und eine Ortsbezeichnung.
- Jeder Passagier hat einen Namen und besitzt Tickets mit einer eindeutigen Nummer.
- Jeder Sitz im Flugzeug hat eine Nummer (nur sinnvoll in Verbindung mit der Flugnummer).
- Zu jedem Sitz ist hinterlegt, ob er für Raucher oder für Nichtraucher gedacht ist.
- Ein Flug wird an genau einem Flugsteig abgefertigt.
- An einem Flugsteig können beliebig viele Flüge abgefertigt werden.
- Eine Ticketnummer ist genau einem Flug zugeordnet.
- Eine Ticketnummer ist genau einem Sitzplatz zugeordnet.
- Ein Sitzplatz kann leer oder von einer Person besetzt sein.
- Ein Sitzplatz ist genau einem Flug zugeordnet.

Übungsaufgabe 2: ERM für einen Hardware-Onlineshop

Erstellen Sie ein ERM für einen Onlineshop, der Computerhardware verkauft. Folgende Annahmen sind zu beachten:

- Jeder Artikel hat eine eindeutige Artikelnummer, eine Bezeichnung und einen Preis.
- Jeder Artikel ist entweder ein Komplettsystem oder eine Komponente, nie beides.
- Komplettsysteme werden als Notebook oder als Desktop-PC angeboten.
- Zu einem Notebook wird die Displaygröße gespeichert, zu einem Desktop-PC der Gehäusetyp.
- Komponenten sind Prozessoren (mit Taktfrequenz) oder Grafikkarten (mit Grafikspeicher).
- Ein Komplettsystem besteht aus mehreren Komponenten.
- Ein Komponentenmodell kann in beliebig vielen Komplettsystemen verbaut sein.
- Komponenten werden auch einzeln verkauft.

Zusatzfrage: Kennzeichnen Sie jede Beziehung als is-a oder is-part-of. Begründen Sie Ihre Wahl mit der Probe „X ist ein Y“ bzw. „X ist Teil von Y“. Handelt es sich beim is-part-of um eine Aggregation oder eine Komposition?

DHBW Stuttgart

Vielen Dank für Ihre Aufmerksamkeit!